FreelanceHub

Piattaforma per la connessione tra Developer Freelancer e Clienti

Indice

1 Dominio Applicativo e Obiettivi **4**
1.1 Contesto . . . 4
1.2 Problema da Risolvere . . . 4
1.3 Soluzione Proposta . . . 4

2 Attori del Sistema **4**
2.1 Guest End User . . . 4
2.2 Cliente . . . 5
2.3 Freelancer . . . 5

3 Terminologia e Glossario **5**

4 Requisiti Funzionali **6**
4.1 Gestione dell'Accesso e dei Profili Utente . . . 6
4.2 Gestione degli Annunci . . . 6
4.3 Gestione delle Candidature . . . 7
4.4 Gestione del Lavoro e della Consegna . . . 7
4.5 Gestione della Presentazione e Validazione . . . 7
4.6 Gestione dei Pagamenti e Accesso ai File . . . 8

5 Requisiti Non Funzionali **8**
5.1 Sicurezza e Privacy . . . 8
5.2 Affidabilità e Disponibilità . . . 9
5.3 Usabilità e Accessibilità . . . 9
5.4 Performance e Scalabilità . . . 9

6 Vincoli Progettuali e Assunzioni **10**
6.1 Vincoli Tecnologici . . . 10
6.2 Vincoli di Business . . . 10
6.3 Assunzioni Operative . . . 10

7 Modello dei Dati **11**
7.1 Entità Principali . . . 12
7.1.1 User . . . 12
7.1.2 Project (Embedded) . . . 12
7.1.3 AnnouncementCandidate . . . 12
7.1.4 Work . . . 13
7.2 UserRole Enumeration . . . 13
7.3 Relazioni . . . 13
7.4 Considerazioni sulla Sicurezza . . . 13

8 State Chart Diagram **14**

9 Regole per gli Smart Contract **14**

10 Casi d'Uso **15**
10.1 Introduzione . . . 15
10.2 Gestione Accesso e Registrazione . . . 15
10.2.1 UC1.1: Effettua Registrazione come Freelancer . . . 15
10.2.2 UC1.2: Effettua Registrazione come Cliente . . . 16
10.2.3 UC2: Effettua Autenticazione . . . 16

10.3 Gestione Annunci 17
10.3.1 UC3: Cliente Pubblica annuncio 17
10.3.2 UC4: Visualizza dettagli Annuncio 17
10.3.3 UC5: Visualizza profilo 18
10.4 Gestione Candidature 18
10.4.1 UC6: Freelancer si Candida per Annuncio 18
10.4.2 UC7: Cliente sceglie candidato 18
10.5 Gestione Lavoro e Consegna 19
10.5.1 UC8: Freelancer carica lavoro 19
10.6 Gestione Presentazione e Validazione 20
10.6.1 UC9: Freelancer presenta lavoro 20
10.6.2 UC9.1: Cliente soddisfatto 20
10.6.3 UC9.2: Cliente richiede modifiche 21
10.6.4 UC9.3: Cliente cambia candidato 21

1. Dominio Applicativo e Obiettivi

1.1 Contesto

La piattaforma è progettata per connettere developer freelancer con clienti che necessitano di servizi di sviluppo software. I clienti possono essere startup, piccole e medie imprese, grandi aziende o agenzie che richiedono competenze tecniche per la realizzazione di progetti software specifici. L'obiettivo primario della piattaforma è ridurre gli attriti lungo l'intero ciclo di ingaggio, dalla scoperta del talento alla conclusione del progetto con il pagamento finale.

La piattaforma si basa su tecnologie Web3 e blockchain, utilizzando wallet MetaMask come meccanismo di autenticazione e identificazione univoca degli utenti. Gli smart contract gestiscono i depositi, l'escrow e i pagamenti, garantendo sicurezza e trasparenza per entrambe le parti. Questo approccio elimina la necessità di credenziali tradizionali e fornisce tracciabilità completa delle transazioni.

Il sistema si propone di semplificare ogni fase del processo di collaborazione, garantendo meccanismi di tutela e trasparenza per entrambe le parti coinvolte. La piattaforma mira a definire tempistiche chiare e a salvaguardare gli interessi sia dei committenti che dei freelancer attraverso un processo strutturato che include scoperta, valutazione, negoziazione, esecuzione, validazione e pagamento.

1.2 Problema da Risolvere

Attualmente, la collaborazione tra freelancer e clienti presenta diverse criticità che la piattaforma intende risolvere. L'assenza di meccanismi di garanzia reciproca genera sfiducia tra le parti. I committenti temono di effettuare pagamenti senza aver verificato la qualità del lavoro svolto, mentre i freelancer rischiano di consegnare il proprio lavoro senza garanzie di compenso. La mancanza di un processo standardizzato per la valutazione e la validazione del lavoro porta a dispute frequenti e incomprensioni sui requisiti. Inoltre, l'assenza di un sistema di tracciamento affidabile rende difficile dimostrare l'autenticità del lavoro presentato.

1.3 Soluzione Proposta

La piattaforma implementa un workflow strutturato che guida le parti attraverso ogni fase della collaborazione. Il sistema prevede meccanismi di protezione basati su smart contract che garantiscono il deposito del compenso e il suo rilascio solo al completamento soddisfacente del lavoro. Un sistema di presentazione controllata permette al cliente di verificare l'autenticità del codice attraverso una demo live con tracciamento dei download. La piattaforma offre inoltre possibilità di iterazione attraverso richieste di modifica o cambio candidato in caso di insoddisfazione, mantenendo sempre la trasparenza e la tutela degli interessi di entrambe le parti.

2. Attori del Sistema

2.1 Guest End User

Il Guest End User rappresenta qualsiasi visitatore non autenticato della piattaforma. Questo attore ha accesso limitato alle funzionalità del sistema e può visualizzare informazioni generali sulla piattaforma. La sua responsabilità principale consiste nell'effettuare la registrazione scegliendo tra il ruolo di Cliente o di Freelancer. Per poter utilizzare la piattaforma, il Guest End User deve disporre obbligatoriamente di un wallet MetaMask installato e configurato nel proprio browser. La registrazione avviene associando i dati dell'utente all'indirizzo del wallet MetaMask, che funge da identificativo univoco. Una volta completata la registrazione, il Guest End User viene automaticamente autenticato attraverso il wallet e acquisisce i privilegi del ruolo selezionato.

2.2 Cliente

Il Cliente è un utente autenticato tramite wallet MetaMask che pubblica annunci di lavoro e assume freelancer per la realizzazione di progetti software. Può rappresentare diverse tipologie di organizzazioni, dalle startup alle grandi enterprise, dalle PMI alle agenzie di comunicazione o sviluppo. Il Cliente è identificato univocamente dal proprio wallet address e da un nickname scelto durante la registrazione. È responsabile della definizione chiara dei requisiti del progetto, della selezione del candidato più idoneo tra quelli che si propongono, della validazione delle consegne ricevute e della decisione finale sul completamento del lavoro. Il Cliente ha inoltre la facoltà di richiedere modifiche al lavoro presentato o di cambiare candidato qualora il risultato non sia soddisfacente. Al termine del processo, effettua il pagamento attraverso il sistema di smart contract integrato nella piattaforma, utilizzando il proprio wallet. Il sistema traccia le statistiche del Cliente, inclusi il numero di lavori pubblicati, il numero di lavori completati e il totale speso.

2.3 Freelancer

Il Freelancer è un professionista autenticato tramite wallet MetaMask che offre servizi di sviluppo software. Durante la registrazione, oltre al nickname, il Freelancer fornisce una descrizione dettagliata delle proprie competenze, una lista di skills tecniche, email di contatto, link al profilo GitHub, link al portfolio personale, e contatti per Discord e Slack. Può inoltre caricare informazioni su progetti precedentemente realizzati, includendo per ciascuno titolo, descrizione e immagini rappresentative. Il Freelancer è identificato univocamente dal proprio wallet address. Naviga la piattaforma alla ricerca di annunci di interesse, valuta le opportunità disponibili e sottomette candidature per i progetti che ritiene in linea con le proprie competenze. Una volta selezionato per un progetto, il Freelancer è responsabile dello sviluppo della soluzione secondo i requisiti specificati nell'annuncio. Deve consegnare il lavoro attraverso la piattaforma caricando i file sorgenti e, successivamente, presentare il risultato al Cliente attraverso una demo live in videochiamata. Il Freelancer ha l'obbligo di illustrare come la soluzione implementata soddisfi i requisiti richiesti e di rispondere alle domande del committente. Il sistema traccia le statistiche del Freelancer, inclusi il numero di lavori pubblicati, il numero di lavori completati e il totale guadagnato.

3. Terminologia e Glossario

Ai fini della comprensione del presente documento, si forniscono le seguenti definizioni dei termini chiave utilizzati nella descrizione del sistema.

Un **Annuncio** rappresenta un'offerta di lavoro pubblicata da un Cliente che non è ancora stata assegnata a nessun Freelancer. L'annuncio contiene la descrizione del problema da risolvere, i requisiti funzionali, le tecnologie coinvolte e il compenso offerto.

Un **Lavoro** è un annuncio che è stato assegnato a un Freelancer specifico e che presenta uno stato di avanzamento tracciato dal sistema. Il lavoro attraversa diverse fasi dalla sua assegnazione fino al completamento o all'eventuale cancellazione.

La **Consegna** indica la submission dei file sorgenti da parte del Freelancer attraverso la piattaforma. La consegna include i file del progetto completato e può contenere commenti opzionali del Freelancer.

La **Presentazione** è una demo live del lavoro svolto che avviene tramite videochiamata esterna alla piattaforma. Durante la presentazione, il Freelancer scarica i file precedentemente consegnati e li esegue in tempo reale, dimostrando al Cliente che la soluzione soddisfa i requisiti specificati.

Una **Candidatura** è la manifestazione di interesse di un Freelancer per un annuncio specifico. Attraverso la candidatura, il Freelancer comunica al Cliente la propria disponibilità a lavorare sul progetto proposto.

4. Requisiti Funzionali

4.1 Gestione dell'Accesso e dei Profili Utente

Il sistema deve consentire la registrazione di nuovi utenti distinguendo tra il ruolo di Freelancer e quello di Cliente (RF01, RF02). Il processo di registrazione richiede obbligatoriamente la connessione di un wallet MetaMask, che funge da meccanismo di autenticazione e identificazione univoca dell'utente attraverso il wallet address.

Durante la registrazione come Freelancer, il sistema richiede nickname, descrizione delle competenze, email, lista di skills tecniche, link GitHub, link portfolio, contatto Discord e contatto Slack. Il Freelancer può inoltre inserire progetti precedenti specificando per ciascuno titolo, descrizione e immagini. Il sistema deve validare che almeno il nickname e una skill siano stati forniti. Per la registrazione come Cliente, il sistema richiede esclusivamente il nickname. In entrambi i casi, dopo l'inserimento dei dati, il sistema richiede la connessione del wallet MetaMask e verifica che il wallet address non sia già associato a un account esistente.

Il sistema deve fornire funzionalità di autenticazione mediante connessione del wallet MetaMask (RF03). Non esistono credenziali tradizionali come email e password; l'autenticazione avviene esclusivamente tramite la firma crittografica del wallet. Quando l'utente richiede l'accesso, il sistema richiede la connessione del wallet MetaMask, ottiene il wallet address e verifica che sia registrato nel sistema. Dopo un'autenticazione riuscita, l'utente viene reindirizzato alla propria homepage personalizzata in base al ruolo. Se il wallet non è registrato, il sistema suggerisce di completare prima la registrazione.

Il sistema deve permettere la visualizzazione dei profili pubblici sia di Clienti che di Freelancer (RF04). Un profilo Freelancer espone wallet address, nickname, descrizione delle competenze, email, lista di skills, link GitHub, link portfolio, contatti Discord e Slack, progetti precedenti con titolo, descrizione e immagini, data di registrazione, numero di lavori pubblicati, numero di lavori completati e totale guadagnato. Un profilo Cliente mostra wallet address, nickname, data di registrazione, numero di lavori pubblicati, numero di lavori completati e totale speso. La visualizzazione del profilo è accessibile da diverse sezioni della piattaforma, come la pagina di dettaglio di un annuncio o la lista dei candidati.

Il sistema deve implementare un meccanismo di notifiche (RF05) che informi gli utenti di eventi rilevanti quali nuove candidature ricevute, assegnazione di un lavoro, consegna completata, richiesta di presentazione e modifiche richieste. Le notifiche possono essere inviate tramite wallet notifications, email (per i Freelancer che hanno fornito l'email) o attraverso canali integrati come Discord e Slack.

4.2 Gestione degli Annunci

Il sistema deve consentire ai Clienti di creare nuovi annunci di lavoro (RF06). Il form di creazione richiede titolo, descrizione dettagliata del problema e dei requisiti, tecnologie utilizzate o da utilizzare, e compenso offerto. Il sistema valida i dati inseriti e, in caso di successo, pubblica l'annuncio rendendolo visibile a tutti i Freelancer.

Il sistema deve fornire una vista che elenca tutti gli annunci pubblicati (RF07). Per ogni annuncio nella lista vengono mostrati titolo, descrizione breve e compenso, permettendo ai Freelancer di avere una panoramica rapida delle opportunità disponibili.

Il sistema deve permettere la visualizzazione dei dettagli completi di un annuncio (RF08). La pagina di dettaglio presenta tutte le informazioni inserite dal Cliente, inclusi titolo, descrizione completa, requisiti, tecnologie e compenso, oltre al nome del Cliente che ha pubblicato l'annuncio.

Il sistema deve gestire lo stato degli annunci (RF09) distinguendo tra annunci aperti alle candidature e annunci assegnati a un Freelancer specifico. Quando un Cliente seleziona un candidato, il sistema cambia automaticamente lo stato dell'annuncio da "aperto" a "assegnato". In caso di cambio candidato, l'annuncio ritorna allo stato "aperto".

4.3 Gestione delle Candidature

Il sistema deve permettere ai Freelancer di inviare candidature per gli annunci di interesse (RF10). Una volta visualizzati i dettagli di un annuncio, il Freelancer può utilizzare la funzionalità di candidatura che registra la sua manifestazione di interesse. Il sistema deve impedire candidature duplicate per lo stesso annuncio.

Il sistema deve fornire ai Clienti la possibilità di visualizzare tutti i candidati che si sono proposti per un loro annuncio (RF11). Per ogni candidato vengono mostrati foto del profilo, nome, cognome e un'anteprima della descrizione delle competenze. Da questa vista, il Cliente può accedere ai profili completi dei candidati per valutarli meglio.

Il sistema deve permettere al Cliente di selezionare e assumere un candidato (RF12). Quando il Cliente utilizza la funzionalità di assunzione, il sistema registra l'assegnazione del lavoro, cambia lo stato dell'annuncio e genera le notifiche appropriate.

Il sistema deve notificare sia il Freelancer selezionato che i candidati non scelti (RF13). Il Freelancer assunto riceve una notifica che lo informa della selezione e può iniziare a lavorare sul progetto. I candidati non selezionati ricevono una comunicazione che li informa dell'esito negativo della loro candidatura.

4.4 Gestione del Lavoro e della Consegna

Il sistema deve fornire una vista dei lavori attivi per i Freelancer (RF14). Questa sezione mostra tutti gli annunci per i quali il Freelancer è stato assunto e che sono attualmente in corso, permettendo di monitorare lo stato di avanzamento di ogni progetto.

Il sistema deve permettere il caricamento di un singolo file per la consegna del lavoro (RF15). Il Freelancer può caricare un file zip contenente tutti i sorgenti del progetto. Il sistema valida il file e ne traccia il caricamento con un timestamp preciso.

Il sistema deve permettere l'aggiunta di commenti opzionali alla consegna (RF16). Prima di confermare il caricamento, il Freelancer può inserire note o spiegazioni riguardo il lavoro svolto che saranno visibili al Cliente.

Il sistema deve gestire la conferma della consegna (RF17). Dopo aver caricato il file e eventualmente aggiunto commenti, il Freelancer deve confermare esplicitamente la consegna. Solo a questo punto il sistema registra la consegna come completata e procede con le notifiche.

Il sistema deve notificare il Cliente quando un Freelancer completa una consegna (RF18). La notifica informa il Cliente che il lavoro è stato consegnato e che può procedere con la richiesta di presentazione.

4.5 Gestione della Presentazione e Validazione

Il sistema deve permettere al Cliente di richiedere la presentazione del lavoro (RF19). Questa funzionalità è disponibile dopo che il Freelancer ha completato la consegna. Quando il Cliente

richiede la presentazione, il sistema blocca la possibilità per il Freelancer di modificare i file consegnati, garantendo l'integrità di quanto verrà presentato.

Il sistema deve abilitare il download dei file durante la presentazione (RF20). Durante la videochiamata, quando il Cliente utilizza la funzionalità "Richiedi Presentazione", il sistema abilita per il Freelancer la possibilità di scaricare i file che aveva precedentemente caricato.

Il sistema deve implementare un contatore di download dei file (RF21). Ogni volta che il Freelancer scarica i file durante la presentazione, il sistema incrementa un contatore visibile al Cliente. Questo meccanismo serve come prova che i file eseguiti durante la demo sono effettivamente quelli consegnati sulla piattaforma.

Il sistema deve gestire la finalizzazione del rapporto in caso di successo (RF22). Quando il Cliente è soddisfatto del lavoro presentato, può utilizzare la funzionalità "Finalizza rapporto con successo" che attiva il pagamento al Freelancer tramite smart contract e fornisce al Cliente l'accesso permanente ai file sorgenti.

Il sistema deve permettere al Cliente di richiedere modifiche (RF23). Se il lavoro presentato non soddisfa completamente i requisiti, il Cliente può specificare quali requisiti non sono stati soddisfatti e può aggiungere commenti esplicativi. Il sistema notifica il Freelancer e sblocca la possibilità di caricare una nuova consegna, riprendendo il ciclo dal caricamento del lavoro.

Il sistema deve permettere al Cliente di cambiare candidato (RF24). Se il Cliente ritiene il lavoro insoddisfacente e non recuperabile, può specificare i requisiti non soddisfatti e procedere con il cambio candidato. Il sistema notifica il Freelancer corrente, chiude l'assegnazione con stato "fallito" e riapre l'annuncio per nuove candidature.

4.6 Gestione dei Pagamenti e Accesso ai File

Il sistema deve gestire la liquidazione del compenso al Freelancer (RF25). Quando il Cliente finalizza il rapporto con successo, il sistema attiva automaticamente il pagamento attraverso lo smart contract, trasferendo la somma depositata dal Cliente al Freelancer.

Il sistema deve fornire al Cliente l'accesso ai file sorgenti dopo il pagamento (RF26). Solo dopo la finalizzazione positiva del rapporto e l'avvenuto pagamento, il Cliente acquisisce la possibilità di scaricare i file sorgenti del progetto.

Il sistema deve gestire il deposito del compenso tramite smart contract (RF27). Al momento della pubblicazione dell'annuncio o dell'assegnazione del candidato, il compenso viene depositato in uno smart contract che lo blocca fino alla finalizzazione del lavoro, garantendo sicurezza per entrambe le parti.

5. Requisiti Non Funzionali

5.1 Sicurezza e Privacy

Il sistema deve garantire che tutte le operazioni sensibili siano accessibili solo ad utenti autenticati tramite wallet MetaMask. L'autenticazione deve essere obbligatoria per qualsiasi azione che vada oltre la semplice visualizzazione di informazioni pubbliche. L'utilizzo di MetaMask come meccanismo di autenticazione fornisce diversi vantaggi di sicurezza: ogni operazione richiede la firma crittografica del wallet, garantendo che solo il possessore della chiave privata possa autenticarsi; non esistono password memorizzate nel sistema, eliminando rischi di leak di credenziali; l'identità dell'utente è verificabile on-chain attraverso il wallet address.

Il sistema deve implementare verifiche per garantire che le operazioni critiche siano effettuate dal wallet address autorizzato. Ad esempio, solo il Cliente proprietario di un annuncio (identificato

dal wallet address) può selezionare candidati o richiedere presentazioni. Solo il Freelancer assegnato a un lavoro può caricare consegne. Queste verifiche devono essere implementate sia lato client che lato server per prevenire tentativi di impersonificazione.

I file caricati dai Freelancer devono essere crittografati e protetti fino al momento del pagamento. Il Cliente non deve avere accesso diretto ai file sorgenti prima della finalizzazione del rapporto con successo e del conseguente pagamento tramite smart contract. Il sistema deve associare i file caricati al wallet address del Freelancer che li ha sottomessi, garantendo tracciabilità e autenticità.

Il sistema deve mantenere un log completo di tutte le operazioni critiche, includendo wallet address dell'utente che ha eseguito l'azione, timestamp, tipo di operazione e hash dei file quando rilevante. Questo registro di audit è fondamentale per garantire la tracciabilità e risolvere eventuali dispute. L'utilizzo di wallet address come identificativi permette inoltre di correlare le operazioni sulla piattaforma con le transazioni on-chain degli smart contract.

5.2 Affidabilità e Disponibilità

Il sistema di notifiche deve essere affidabile e garantire la consegna tempestiva delle comunicazioni agli utenti. Eventi critici come nuove candidature, assegnazioni di lavori e completamento di consegne devono generare notifiche che raggiungano gli utenti interessati senza ritardi significativi.

I file caricati dai Freelancer devono essere sottoposti a backup automatico per prevenire perdite di dati. Il sistema deve garantire la persistenza dei file anche in caso di guasti hardware o problemi tecnici.

Tutte le transazioni relative a caricamenti, consegne, richieste di presentazione e pagamenti devono essere registrate in modo permanente e immutabile. Il sistema deve garantire che queste informazioni siano sempre disponibili e consultabili per scopi di audit o risoluzione di dispute.

5.3 Usabilità e Accessibilità

L'interfaccia utente deve essere progettata per essere intuitiva anche per utenti con competenze tecniche limitate. Considerando che molti Clienti potrebbero non avere un background informatico, il sistema deve guidarli attraverso ogni fase del processo con istruzioni chiare e messaggi di errore comprensibili.

Il sistema deve fornire dashboard chiare che permettano sia ai Clienti che ai Freelancer di monitorare facilmente lo stato dei loro annunci e lavori. Le informazioni più rilevanti devono essere facilmente accessibili e le azioni più comuni devono richiedere un numero minimo di click.

Il sistema dovrebbe supportare almeno due lingue, italiano e inglese, per permettere una maggiore accessibilità a utenti internazionali.

5.4 Performance e Scalabilità

I tempi di caricamento delle pagine devono essere contenuti, con un obiettivo di meno di due secondi per la maggior parte delle operazioni di navigazione. Operazioni più complesse come il caricamento di file possono richiedere tempi maggiori, ma devono comunque fornire feedback visuale all'utente durante l'elaborazione.

Il sistema deve supportare il caricamento di file di dimensioni ragionevoli, fino a 500MB, per permettere la consegna di progetti di media complessità senza necessità di soluzioni esterne per il trasferimento file.

Il sistema deve essere progettato per gestire in modo efficiente almeno 1000 utenti concorrenti senza degradazione significativa delle performance.

6. Vincoli Progettuali e Assunzioni

6.1 Vincoli Tecnologici

Il sistema richiede obbligatoriamente che gli utenti dispongano di un wallet MetaMask installato e configurato nel proprio browser. MetaMask funge da meccanismo di autenticazione, identificazione e interfaccia per le transazioni blockchain. Senza MetaMask, non è possibile registrarsi o autenticarsi sulla piattaforma. Questo vincolo implica che il sistema sia progettato primariamente per utenti desktop o mobile che utilizzano browser compatibili con estensioni MetaMask.

Il sistema di pagamento deve essere implementato utilizzando smart contract per garantire l'atomicità delle transazioni e la protezione del deposito fino alla finalizzazione del lavoro. Gli smart contract devono interagire con i wallet degli utenti per gestire depositi, escrow e rilasci di pagamento. Ogni transazione richiede la firma esplicita del wallet address del Cliente per i depositi e viene automaticamente eseguita verso il wallet address del Freelancer al completamento con successo.

Il sistema deve implementare un meccanismo di storage sicuro per i file caricati dai Freelancer. I file devono essere crittografati e associati al wallet address del Freelancer che li ha caricati. L'accesso deve essere controllato dal sistema in base allo stato del lavoro, al wallet address richiedente e alla finalizzazione del pagamento.

Il sistema deve tracciare in modo immutabile il contatore di download dei file durante le presentazioni, registrando il wallet address dell'utente che effettua il download e il timestamp. Questo meccanismo di audit è essenziale per garantire la trasparenza del processo di presentazione e fornire evidenze verificabili in caso di dispute.

6.2 Vincoli di Business

Il modello di business prevede che la piattaforma applichi una commissione su ogni transazione completata con successo. I dettagli esatti della percentuale di commissione e delle modalità di applicazione dovranno essere definiti nelle fasi successive del progetto.

Il sistema dovrà includere politiche chiare per la gestione delle dispute tra Clienti e Freelancer. Sebbene il sistema attuale preveda meccanismi di richiesta modifiche e cambio candidato, potrebbero essere necessari processi di arbitrato per situazioni più complesse.

Il sistema deve rispettare tutti i requisiti legali relativi ai contratti di lavoro freelance e ai pagamenti online, incluse eventuali normative fiscali applicabili.

6.3 Assunzioni Operative

Si assume che la comunicazione dettagliata tra Cliente e Freelancer avvenga attraverso canali esterni alla piattaforma, come email o telefono. La piattaforma fornisce i contatti necessari ma non gestisce direttamente la messaggistica tra le parti.

Si assume che le presentazioni del lavoro avvengano tramite strumenti di videoconferenza esterni alla piattaforma. Il sistema fornisce il supporto per la gestione del download controllato dei file durante la presentazione, ma non integra funzionalità di videochiamata.

Si assume che il Cliente sia responsabile della verifica durante la presentazione. Il sistema fornisce meccanismi di supporto come il contatore di download, ma la responsabilità finale di accertarsi che il codice presentato sia quello consegnato resta al Cliente.

Si assume che i file consegnati siano in formato compresso (zip) quando contengono molteplici file sorgenti. Il sistema gestisce il caricamento di un singolo file per semplificare il processo e ridurre la complessità.

7. Modello dei Dati

Il modello dei dati della piattaforma definisce la struttura delle entità principali e le loro relazioni. La Figura 2 mostra lo schema concettuale del database.

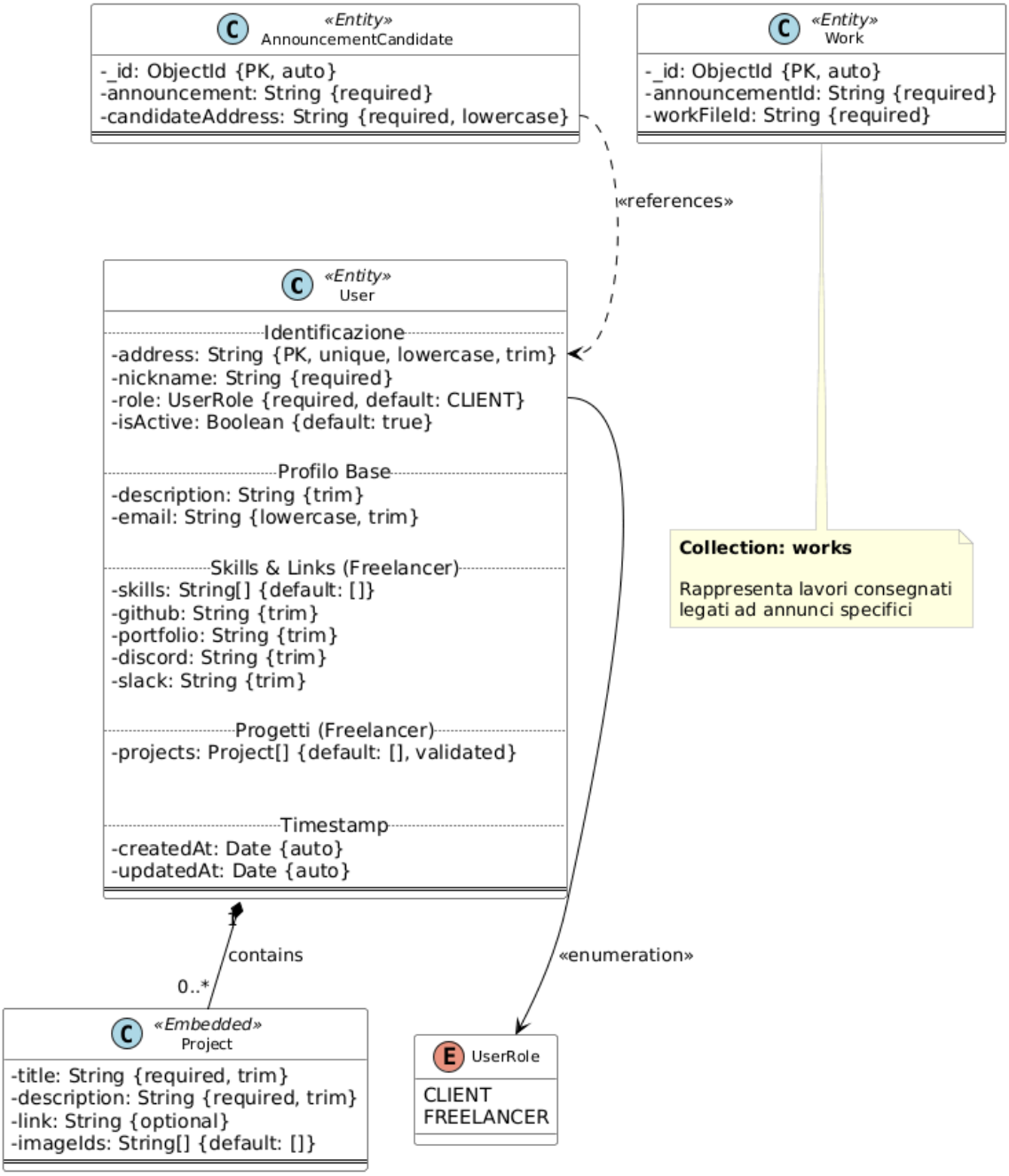

Figura 1: Schema del modello dei dati

7.1 Entità Principali

7.1.1 User

L'entità **User** rappresenta tutti gli utenti della piattaforma, sia Clienti che Freelancer. L'entità è strutturata nelle seguenti sezioni:

Identificazione:

- `address`: Indirizzo del wallet MetaMask, utilizzato come chiave primaria. È univoco, lowercase e viene trimmed per evitare inconsistenze.
- `nickname`: Nome scelto dall'utente (required).
- `role`: Ruolo dell'utente, che può essere CLIENT o FREELANCER (enum UserRole). Il valore di default è CLIENT.
- `isActive`: Flag booleano che indica se l'account è attivo (default: true).

Profilo Base:

- `description`: Descrizione delle competenze (trimmed, opzionale).
- `email`: Indirizzo email (lowercase, trimmed, opzionale).

Skills & Links (Freelancer):

- `skills`: Array di stringhe contenente le competenze tecniche (default: array vuoto).
- `github`: Link al profilo GitHub (trimmed, opzionale).
- `portfolio`: Link al portfolio personale (trimmed, opzionale).
- `discord`: Contatto Discord (trimmed, opzionale).
- `slack`: Contatto Slack (trimmed, opzionale).

Progetti (Freelancer):

- `projects`: Array di oggetti Project embedded (default: array vuoto, validated).

Timestamp:

- `createdAt`: Data di creazione dell'account (auto).
- `updatedAt`: Data di ultimo aggiornamento (auto).

7.1.2 Project (Embedded)

L'entità **Project** è embedded all'interno di User e rappresenta i progetti precedenti realizzati dai Freelancer:

- `title`: Titolo del progetto (required, trimmed).
- `description`: Descrizione del progetto (required, trimmed).
- `link`: Link al progetto (opzionale).
- `imageIds`: Array di identificatori delle immagini (default: array vuoto).

7.1.3 AnnouncementCandidate

L'entità **AnnouncementCandidate** rappresenta le candidature dei Freelancer agli annunci:

- `id`: Identificatore univoco (ObjectId, PK, auto).
- `announcement`: Riferimento all'annuncio (String, required).
- `candidateAddress`: Indirizzo wallet del candidato (String, required, lowercase).

Questa entità stabilisce la relazione many-to-many tra annunci e candidati.

7.1.4 Work

L'entità **Work** rappresenta i lavori assegnati e la loro evoluzione:

- `id`: Identificatore univoco (ObjectId, PK, auto).
- `announcementId`: Riferimento all'annuncio associato (String, required).
- `workField`: Campo descrittivo del lavoro (String, required).

L'entità Work mantiene la collection `works` che rappresenta i lavori consegnati legati ad annunci specifici. Attraverso questa entità è possibile tracciare lo stato di avanzamento dei progetti, le consegne effettuate e le relazioni tra annunci e lavori completati.

7.2 UserRole Enumeration

L'enumerazione **UserRole** definisce i ruoli possibili per gli utenti:

- `CLIENT`: Identifica un Cliente che pubblica annunci e assume Freelancer.
- `FREELANCER`: Identifica un Freelancer che si candida e lavora su progetti.

7.3 Relazioni

Il modello prevede le seguenti relazioni principali:

- Un **User** può contenere zero o più **Project** embedded (0..*).
- Un **AnnouncementCandidate** referenzia un **User** tramite il campo `candidateAddress` (relazione "references").
- La collection **works** in **Work** contiene i lavori associati agli annunci.

7.4 Considerazioni sulla Sicurezza

Il modello utilizza l'indirizzo del wallet MetaMask come identificatore univoco e chiave primaria per gli utenti. Questo approccio garantisce:

- Identificazione univoca e non ambigua degli utenti.
- Autenticazione crittografica tramite firma del wallet.
- Tracciabilità delle operazioni on-chain.
- Eliminazione della necessità di memorizzare password tradizionali.

L'uso di `lowercase` e `trim` sui campi critici come `address`, `email` e `candidateAddress` previene inconsistenze dovute a variazioni di formato.

8. State Chart Diagram

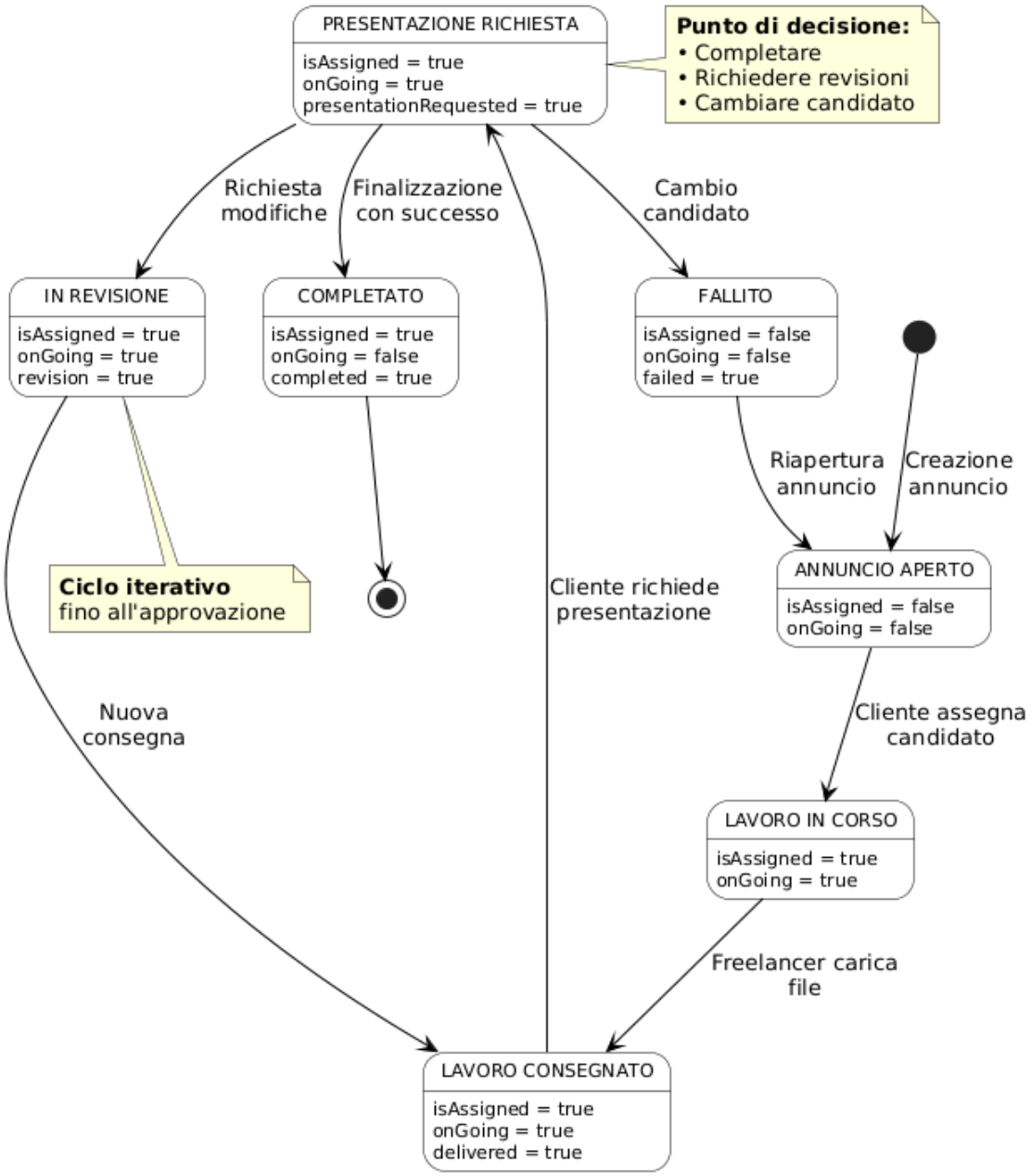

Figura 2: Diagramma degli stati di un lavoro

9. Regole per gli Smart Contract

Gli smart contract implementano le regole di business critiche che garantiscono la correttezza delle transazioni e la tutela degli interessi di entrambe le parti.

La **regola di blocco della consegna** stabilisce che dopo la richiesta di presentazione da parte del Cliente, il Freelancer non può più modificare i file consegnati. Questa regola garantisce l'integrità di quanto verrà presentato e impedisce manipolazioni dell'ultimo momento.

La **regola del timeout di presentazione** prevede che se il Cliente non richiede la presentazione entro un numero prestabilito di giorni dalla consegna, il Freelancer può richiedere il pagamento

automatico. Questa protezione impedisce che i Clienti ritardino indefinitamente la validazione del lavoro.

La **regola del tracciamento download** garantisce che ogni operazione di download durante la presentazione venga registrata in modo immutabile. Il registro include timestamp, identificativo dell'utente che ha effettuato il download e hash del file scaricato, fornendo evidenze oggettive in caso di dispute.

La **regola dell'escrow payment** stabilisce che il compenso viene bloccato in uno smart contract al momento della pubblicazione dell'annuncio o dell'assegnazione del candidato. Il pagamento rimane bloccato fino all'approvazione del Cliente o alla scadenza del timeout per le dispute. Solo in questi casi il trasferimento al Freelancer viene eseguito automaticamente, garantendo che il compenso sia sempre disponibile e protetto.

10. Casi d'Uso

10.1 Introduzione

Attori: Guest End User, Cliente, Freelancer.

La presente sezione descrive in dettaglio i casi d'uso del sistema, organizzati per funzionalità. Ogni caso d'uso specifica l'attore coinvolto, le condizioni di ingresso, il flusso principale di eventi, le condizioni di uscita e gli eventuali flussi alternativi o eccezionali.

10.2 Gestione Accesso e Registrazione

10.2.1 UC1.1: Effettua Registrazione come Freelancer

Attore: Guest End User

Entry conditions: L'utente si trova sulla pagina di registrazione e dispone di un wallet MetaMask

Flusso di eventi:

1. L'utente sceglie di registrarsi come Freelancer
2. L'utente inserisce nickname, descrizione delle competenze, email, skills (lista di competenze), link GitHub, link portfolio, contatto Discord, contatto Slack
3. L'utente può opzionalmente inserire progetti precedenti specificando per ciascuno titolo, descrizione e immagini
4. L'utente sottomette i dati al sistema
5. Il sistema verifica che tutti i campi obbligatori siano compilati (nickname e almeno una skill)
6. Il sistema verifica il formato dell'email (se fornita)
7. Il sistema richiede la connessione del wallet MetaMask
8. L'utente conferma la connessione del wallet MetaMask
9. Il sistema ottiene l'indirizzo del wallet
10. Il sistema verifica che l'indirizzo wallet non sia già registrato
11. Il sistema crea l'account associando i dati al wallet address
12. Il sistema autentica automaticamente l'utente
13. Il sistema reindirizza l'utente alla homepage

Exit conditions: L'utente si trova autenticato sulla homepage.

Flussi alternativi/eccezionali:

- Se al punto 5 o 6 i controlli falliscono, il sistema mostra messaggi di errore specifici e presenta nuovamente la schermata di registrazione con i dati già inseriti. L'utente può correggere gli errori e riprovare.
- Se al punto 8 l'utente rifiuta la connessione del wallet, il sistema mostra il messaggio "Connessione wallet necessaria per completare la registrazione" e interrompe il processo.
- Se al punto 10 il wallet è già registrato, il sistema mostra il messaggio "Wallet già registrato" e suggerisce di effettuare l'accesso [UC1.1.1 Registrazione Fallita].
- Se l'utente non dispone di MetaMask installato, il sistema mostra un messaggio che suggerisce di installare l'estensione MetaMask.

10.2.2 UC1.2: Effettua Registrazione come Cliente

Attore: Guest End User

Entry conditions: L'utente si trova sulla pagina di registrazione e dispone di un wallet MetaMask

Flusso di eventi:

1. L'utente sceglie di registrarsi come Cliente
2. L'utente inserisce nickname
3. L'utente sottomette i dati al sistema
4. Il sistema verifica che il nickname sia stato inserito
5. Il sistema richiede la connessione del wallet MetaMask
6. L'utente conferma la connessione del wallet MetaMask
7. Il sistema ottiene l'indirizzo del wallet
8. Il sistema verifica che l'indirizzo wallet non sia già registrato
9. Il sistema crea l'account associando il nickname al wallet address
10. Il sistema autentica automaticamente l'utente
11. Il sistema reindirizza l'utente alla homepage

Exit conditions: L'utente si trova autenticato sulla homepage.

Flussi alternativi/eccezionali:

- Se al punto 4 il nickname non è stato inserito, il sistema mostra un messaggio di errore e presenta nuovamente la schermata di registrazione.
- Se al punto 6 l'utente rifiuta la connessione del wallet, il sistema mostra il messaggio "Connessione wallet necessaria per completare la registrazione" e interrompe il processo.
- Se al punto 8 il wallet è già registrato, il sistema mostra il messaggio "Wallet già registrato" e suggerisce di effettuare l'accesso [UC1.2.1 Registrazione Fallita].
- Se l'utente non dispone di MetaMask installato, il sistema mostra un messaggio che suggerisce di installare l'estensione MetaMask.

10.2.3 UC2: Effettua Autenticazione

Attore: Freelancer, Cliente

Entry conditions: L'utente visualizza la pagina di login e dispone di un wallet MetaMask con un account già registrato

Flusso di eventi:

1. L'utente utilizza la funzionalità "Connetti Wallet"
2. Il sistema richiede la connessione del wallet MetaMask
3. L'utente conferma la connessione del wallet MetaMask
4. Il sistema ottiene l'indirizzo del wallet

5. Il sistema verifica che l'indirizzo wallet sia registrato
6. Il sistema autentica l'utente identificandolo tramite il wallet address
7. Il sistema reindirizza l'utente alla sua homepage

Exit conditions: L'utente è autenticato e si trova sulla sua home page.

Flussi alternativi/eccezionali:

- Se al punto 3 l'utente rifiuta la connessione del wallet, il sistema mostra il messaggio "Connessione wallet necessaria per accedere" e presenta nuovamente la schermata di login.
- Se al punto 5 il sistema rileva che il wallet non è registrato, il sistema mostra il messaggio "Wallet non registrato. Effettua prima la registrazione" e presenta la schermata di registrazione [UC2.1 Autenticazione Fallita].
- Se l'utente non dispone di MetaMask installato, il sistema mostra un messaggio che suggerisce di installare l'estensione MetaMask.

10.3 Gestione Annunci

10.3.1 UC3: Cliente Pubblica annuncio

Attore: Cliente

Entry conditions: L'utente è autenticato e visualizza la sua homepage

Flusso di eventi:

1. L'utente utilizza la funzionalità Crea un annuncio
2. Il sistema reindirizza l'utente alla pagina di creazione di un annuncio
3. L'utente inserisce titolo, descrizione e requisiti del problema, tecnologie utilizzate / da utilizzare, compenso
4. L'utente sottomette i dati al sistema
5. Il sistema valida i dati inseriti
6. Il sistema crea l'annuncio
7. Il sistema reindirizza l'utente alla pagina personale degli annunci pubblicati

Exit conditions: L'utente si ritrova alla pagina personale degli annunci pubblicati e visualizza l'annuncio appena pubblicato.

Flussi alternativi/eccezionali:

- Se al punto 5 la validazione fallisce, il sistema mostra errori specifici e presenta nuovamente il form con i dati già inseriti. L'utente può correggere e riprovare.

10.3.2 UC4: Visualizza dettagli Annuncio

Attore: Cliente / Freelancer

Entry conditions: L'utente è autenticato e visualizza la pagina degli annunci pubblicati.

Flusso di eventi:

1. L'utente visualizza la lista degli annunci pubblicati; per ogni annuncio visualizza titolo, descrizione (breve) e compenso
2. L'utente sceglie un annuncio dalla lista degli annunci pubblicati
3. L'utente utilizza la funzionalità Visualizza dettagli in corrispondenza di esso
4. Il sistema reindirizza l'utente alla pagina di dettaglio dell'annuncio
5. Il sistema mostra tutte le informazioni complete dell'annuncio: titolo, descrizione completa e requisiti, tecnologie, compenso, nome del Cliente che ha pubblicato l'annuncio

Exit conditions: L'utente visualizza la pagina di dettaglio dell'annuncio.

10.3.3 UC5: Visualizza profilo

Attore: Cliente / Freelancer

Entry conditions: L'utente è autenticato

Flusso di eventi:

1. L'utente si trova su una pagina dove è presente un link al profilo di un altro utente (es. dalla pagina dettagli annuncio, dalla lista candidati)
2. L'utente utilizza la funzionalità Visualizza profilo
3. Il sistema reindirizza l'utente alla pagina del profilo
4. Il sistema mostra le informazioni pubbliche del profilo

Exit conditions: L'utente visualizza il profilo pubblico di un altro utente.

Note:

- Per un profilo Freelancer il sistema mostra: wallet address, nickname, descrizione delle competenze, email, skills (lista competenze), link GitHub, link portfolio, contatti Discord, contatti Slack, progetti precedenti (con titolo, descrizione e immagini), data di registrazione, numero di lavori pubblicati, numero di lavori completati, totale guadagnato.
- Per un profilo Cliente il sistema mostra: wallet address, nickname, data di registrazione, numero di lavori pubblicati, numero di lavori completati, totale speso.
- Un Cliente può visionare il profilo di un Freelancer e viceversa.

10.4 Gestione Candidature

10.4.1 UC6: Freelancer si Candida per Annuncio

Attore: Freelancer

Entry conditions: L'utente ha eseguito UC4: Visualizza dettagli Annuncio

Flusso di eventi:

1. L'utente utilizza la funzionalità Manda candidatura
2. Il sistema registra la candidatura
3. Il sistema produce il messaggio "Candidatura effettuata"
4. Il sistema notifica il Cliente della nuova candidatura

Exit conditions: L'utente si è candidato per il lavoro offerto dal Cliente; il Cliente viene notificato per la nuova candidatura.

Flussi alternativi/eccezionali:

- Se al punto 1 l'utente ha già inviato candidatura per questo annuncio, il sistema mostra il messaggio "Hai già inviato candidatura per questo annuncio" e non permette candidature duplicate.

10.4.2 UC7: Cliente sceglie candidato

Attore: Cliente

Entry conditions: L'utente è autenticato.

Flusso di eventi:

1. L'utente esegue UC4: Visualizza dettagli Annuncio relativamente a un annuncio pubblicato
2. L'utente utilizza la funzionalità Visualizza candidati relativamente all'annuncio scelto
3. Il sistema reindirizza l'utente alla pagina di tutti i freelancer che si sono candidati
4. Il sistema mostra per ogni candidato: nickname, breve descrizione delle competenze (prime righe) e principali skills
5. L'utente sceglie un freelancer utilizzando la funzionalità Assumi candidato
6. Il sistema registra l'assegnazione
7. L'utente visualizza il messaggio "Freelancer assunto"
8. Il sistema notifica il Freelancer scelto
9. Il sistema notifica i candidati non selezionati
10. Il sistema cambia lo stato dell'annuncio da "aperto" a "assegnato"

Exit conditions: L'utente freelancer che è stato assunto viene notificato di essere stato scelto per il lavoro da svolgere.

Flussi alternativi/eccezionali:

- Al punto 1 l'utente può accedere alla lista dei candidati a un annuncio pubblicato attraverso il link relativo a una notifica ricevuta.
- Al punto 5, prima di utilizzare la funzionalità Assumi candidato, l'utente può eseguire UC5 Visualizza profilo relativamente al possibile freelancer da scegliere.

10.5 Gestione Lavoro e Consegna

10.5.1 UC8: Freelancer carica lavoro

Attore: Freelancer

Entry conditions: L'utente è autenticato ed ha annunci attivi su cui lavora

Flusso di eventi:

1. L'utente esegue UC4: Visualizza dettagli Annuncio relativamente all'annuncio su cui ha finito di lavorare
2. L'utente utilizza la funzionalità Carica lavoro e carica il file relativo al lavoro svolto
3. Il sistema valida il file
4. L'utente visualizza il file caricato
5. L'utente sottomette i dati al sistema utilizzando la funzionalità Conferma
6. Il sistema registra la consegna con timestamp
7. Il sistema mostra all'utente il messaggio di avvenuta consegna del lavoro
8. Il sistema notifica il Cliente dell'avvenuta consegna

Exit conditions:

- Il Freelancer visualizza il messaggio di avvenuta consegna del lavoro.
- Il Cliente, titolare dell'annuncio per il quale il Freelancer ha consegnato il lavoro, viene notificato dell'avvenuta consegna del lavoro.

Flussi alternativi/eccezionali:

- Al punto 5, prima di confermare la consegna, l'utente può aggiungere un commento riguardo la consegna che sta per effettuare.
- Se al punto 2 l'utente prova a caricare più file il sistema mostra il messaggio: "È possibile caricare un singolo file".

Vincoli:

- È possibile caricare un singolo file (usare formato .zip per progetti con più file).

- Non è possibile modificare la consegna dopo che il Cliente ha richiesto la presentazione.

10.6 Gestione Presentazione e Validazione

10.6.1 UC9: Freelancer presenta lavoro

Attori: Freelancer e Cliente

Entry conditions:

- Freelancer ha eseguito UC8: Freelancer carica lavoro;
- Freelancer e Cliente sono in videochiamata (accordata in precedenza tramite canali terzi) e il Freelancer condivide lo schermo al Cliente.
- Freelancer e Cliente eseguono UC4: Visualizza dettagli Annuncio in corrispondenza del lavoro (annuncio) per il quale si sono accordati (quindi entrambi si trovano alla pagina di dettaglio dell'annuncio).

Flusso di eventi:

1. Cliente utilizza la funzionalità Richiedi Presentazione
2. Il sistema registra la richiesta di presentazione
3. Il sistema blocca la possibilità per il Freelancer di modificare la consegna
4. Il sistema mostra un messaggio di successo e come side effect permette al Freelancer la possibilità di usare la funzionalità Scarica File
5. Freelancer utilizza la funzionalità Scarica File
6. Il sistema incrementa di 1 il contatore di download del file (il Cliente ha così la prova effettiva che il lavoro scaricato che andrà a presentare sia quello effettivamente consegnato) e come side effect il Cliente ha accesso alle funzionalità Promuovi, Richiedi modifiche e Cambia candidato
7. Freelancer illustra al Cliente il lavoro svolto: esegue il codice scaricato, dimostra come il lavoro soddisfa i requisiti dell'annuncio, risponde alle domande del Cliente
8. Cliente osserva la presentazione e verifica che i file scaricati ed eseguiti dal Freelancer corrispondano e che il contatore download sia stato incrementato
9. Cliente utilizza una delle tre funzionalità a sua disposizione: Promuovi, Richiedi modifiche o Cambia candidato

Exit conditions: Il Cliente prende visione del lavoro effettuato dal Freelancer.

Flussi alternativi/eccezionali:

- Se al punto 9 il Cliente è soddisfatto del lavoro visionato, utilizzando la funzionalità Promuovi si verifica UC9.1 Cliente soddisfatto;
- Se il Cliente non è soddisfatto e vuole richiedere delle modifiche al lavoro effettuato, utilizzando la funzionalità Richiedi modifiche, si verifica UC9.2 Cliente richiede modifiche.
- Se il Cliente desidera cambiare candidato ed utilizza la funzionalità Cambia candidato, si verifica UC9.3 Cliente cambia candidato.

Note: Dopo il punto 4, il Freelancer NON può più modificare la consegna effettuata. Il contatore download serve come prova che il file presentato è quello consegnato.

10.6.2 UC9.1: Cliente soddisfatto

Attore: Cliente

Entry conditions:

- Freelancer e Cliente hanno eseguito UC9: Freelancer presenta lavoro;

- Cliente è soddisfatto del lavoro svolto;
- Cliente esegue UC4: Visualizza dettagli Annuncio

Flusso di eventi:

1. Cliente utilizza la funzionalità Finalizza rapporto con successo
2. Come side effect la quantità depositata nel contratto va liquidata al freelancer e il Cliente avrà la nuova funzionalità in corrispondenza dell'annuncio, Scarica file
3. Il sistema chiude il lavoro con stato "completato"
4. Il sistema aggiorna la reputazione del Freelancer
5. Cliente utilizza la funzionalità Scarica file
6. Il sistema fornisce accesso ai file sorgenti al Cliente
7. Il sistema conferma l'operazione conclusa
8. Il sistema notifica il Freelancer del pagamento avvenuto

Exit conditions: Il Freelancer viene pagato, il Cliente riceve l'accesso al codice sorgente.

10.6.3 UC9.2: Cliente richiede modifiche

Attore: Cliente

Entry conditions:

- Freelancer e Cliente hanno eseguito UC9: Freelancer presenta lavoro;
- Cliente non è soddisfatto del lavoro svolto, ma vuole proseguire con lo stesso candidato richiedendo di fare modifiche necessarie;
- Cliente esegue UC4: Visualizza dettagli Annuncio

Flusso di eventi:

1. Cliente utilizza la funzionalità Richiedi modifiche
2. Il sistema presenta un form per specificare le modifiche
3. Cliente specifica quali dei requisiti iniziali dell'annuncio non sono stati soddisfatti
4. Cliente sottomette le richieste
5. Il sistema registra le richieste di modifica
6. Il sistema sblocca la possibilità per il Freelancer di caricare una nuova consegna
7. Il sistema notifica il Freelancer con i dettagli delle modifiche richieste
8. Il sistema cambia lo stato del lavoro in "in revisione"

Exit conditions: L'utente freelancer viene notificato delle modifiche richieste relative all'annuncio presentato.

Flussi alternativi/eccezionali:

- Al punto 3, Cliente può lasciare un commento sul perché un certo requisito non è soddisfatto.

Note: Il ciclo ritorna a UC8 (Freelancer carica lavoro).

10.6.4 UC9.3: Cliente cambia candidato

Attore: Cliente

Entry conditions:

- Freelancer e Cliente hanno eseguito UC9: Freelancer presenta lavoro;
- Cliente non è soddisfatto del lavoro svolto e vuole cambiare candidato;
- Cliente esegue UC4: Visualizza dettagli Annuncio

Flusso di eventi:

1. Cliente utilizza la funzionalità Cambia candidato
2. Il sistema presenta un form per specificare i motivi
3. Cliente specifica quali dei requisiti iniziali dell'annuncio non sono stati soddisfatti
4. Cliente sottomette la richiesta
5. Il sistema notifica il Freelancer che non è più incaricato del lavoro
6. Il sistema chiude l'assegnazione corrente con stato "fallito"
7. Il sistema riapre l'annuncio per nuove candidature
8. Il sistema cambia lo stato dell'annuncio da "assegnato" a "aperto"

Exit conditions: L'utente freelancer viene notificato che non è più incaricato come candidato del lavoro assegnato e si procede con attesa dei nuovi candidati per tale annuncio

Flussi alternativi/eccezionali:

- Al punto 3, Cliente può lasciare un commento sul perché un certo requisito non è soddisfatto.

Note:

- Questo scenario implica che il lavoro del Freelancer precedente non viene pagato.
- Il ciclo ritorna a UC7 (Cliente sceglie candidato).